

POLITECHNIKA CZĘSTOCHOWSKA
Katedra InformatykiLaboratorium z przedmiotu
Projekt zespołowy

KARTA PROJEKTU

GRUPA: 2
Zespół IO_1Rok akademicki
2026/2027

1. Temat projektu:

Wewnętrzny system zarządzania procesami sądowymi
Hiszpańskiej Inkwizycji

2. Kierownik projektu:

Maksym Domilovskyi



3. Skład zespołu:

Imię i nazwisko	Główne zadanie
Maksym Domilovskyi	FullStack
Roman Potapov	Backend
Yehor Zaivenko	Frontend

4. Opis projektu:

Projekt zakłada stworzenie rozproszonego systemu informatycznego w architekturze klient-serwer, służącego do ewidencjonowania tajnych akt sądowych. Ponieważ nikt nie spodziewa się Hiszpańskiej Inkwizycji, system musi działać szybko i bezwzględnie. Aplikacja kliencka zostanie napisana w boskim języku Holy C, natomiast backend - pełniący rolę Głównego Inkwizytora logiki biznesowej - w języku Java, przechowując dane o grzechach w bazie PostgreSQL.

4.1 Cel projektu:

Naszą Świętą Misją jest ułatwienie pracy inkwizytorom poprzez cyfryzację zapasów pergaminu i zautomatyzowanie procesu śledzenia przewinień oraz wydawania wyroków na heretyków.

4.2 Główne założenia:

(dla każdego założenia/zadania stworzyć DoD (Definition of Done))

- Architektura klient-serwer. (DoD: Serwer Java i klient w Holy C potrafią nawiązać poprawne połączenie i wymieniać dane o sprawach).
- Centralny rejestr w PostgreSQL. (DoD: Backend poprawnie łączy się z bazą i wykonuje podstawowe operacje odczytu oraz zapisu akt).
- Interfejs w języku Holy C. (DoD: Interfejs w języku Holy C pozwala na wpisanie danych heretyka i pomyślnie przesłanie ich na ołtarz serwera).

5. Wymagania funkcjonalne:

5.1 Podstawowe funkcjonalności (must-have):

Nazwa funkcjonalności	Kryteria akceptacji
Rejestracja heretyka	Użytkownik może dodać nową sprawę, wypełniając pola: imię, nazwisko, wiek, czy chrzczony (tak/nie), rodzaj herezji oraz wyrok. Zapis pomyślnie trafia do bazy.
Zarządzanie przesłuchaniami	Możliwość ustalenia terminu, miejsca i metod "przesłuchania".
Przeglądanie rejestru grzechów	Klient pobiera z serwera i wyświetla pełną listę zapisanych uprzednio spraw.

5.2 Opcjonalne funkcjonalności (nice-to-have):

Nazwa funkcjonalności	Kryteria akceptacji
Statystyki stosów	Proste podsumowanie zliczające wydane wyroki.
Inwentarz sali przesłuchań	Możliwość sprawdzenia dostępności narzędzi.
Filtrowanie akt	Możliwość wyszukiwania na liście konkretnych grzeszników po rodzaju herezji.

6. Wymagania нефункционалне:

- Aplikacja kliencka napisana w języku Holy C
- Aplikacja serwerowa zaimplementowana w języku Java.
- Wykorzystanie relacyjnej bazy danych PostgreSQL.
- Brak rygorystycznych ograniczeń wydajnościowych.

7. Harmonogram prac:

Data zajęć	Opis realizowanych zagadnień
5.10.2026	Rozpoczęcie prac, podział ról, ustalenie wymagań i wybór narzędzi.
12.10.2026	Prace nad ogólną architekturą systemu i schematem bazy danych.
19.10.2026	Konfiguracja szkieletu serwera w Javie i bazy PostgreSQL.
26.10.2026	Pierwsze testy składni Holy C.
2.11.2026	Prace nad podstawowym interfejsem użytkownika we frontendzie.
9.11.2026	Tworzenie punktów końcowych (endpointów/gniazd) po stronie serwera Java.
16.11.2026	Próby zestawienia komunikacji sieciowej między Frontendem a Javą.
23.11.2026	Implementacja funkcjonalności przesyłania formularza nowej sprawy.
30.11.2026	Implementacja funkcjonalności pobierania i wyświetlania list spraw.
7.12.2026	Ogólna integracja systemów, łączenie i testowanie klient-serwer.
14.12.2026	Opcjonalne prace nad funkcjonalnościami dodatkowymi (filtrowanie, statystyki).
21.12.2027	Weryfikacja działania całego systemu i naprawa błędów.
11.01.2027	Ostateczne testy, refaktoryzacja i przygotowanie dokumentacji.
18.01.2027	Prezentacje końcowych wersji projektów, oddanie dokumentacji.
25.01.2027	Końcowe zaliczenie projektu

8. Zarządzenie ryzykiem:

8.1 Identyfikacja ryzyk

- Całkowity brak wcześniejszego doświadczenia zespołu z językiem Holy C.
- Brak zewnętrznych bibliotek i obszernych poradników do rozwiązywania problemów we frontendzie.

8.2. Plan reagowania

Szybkie stworzenie najprostszego prototypu (Proof of Concept) komunikacji między Javą a Holy C, aby upewnić się o wykonalności zadania, zanim powstaną zaawansowane moduły.

9. Dokumentacja:

9.1 Dokumentacja techniczna (Doxygen)

.....

9.2 Dokumentacja użytkowa

.....

9.3 Dokumentacja wdrożeniowa

.....

Link do Jiry:

Link do GitHub:
